birdclef-2026

BirdCLEF+2026 18th Place Solution: Ensemble is all we need

Rank	18
Team	Win or lose?
Author	cjwing
Topic ID	704287
Version	Original

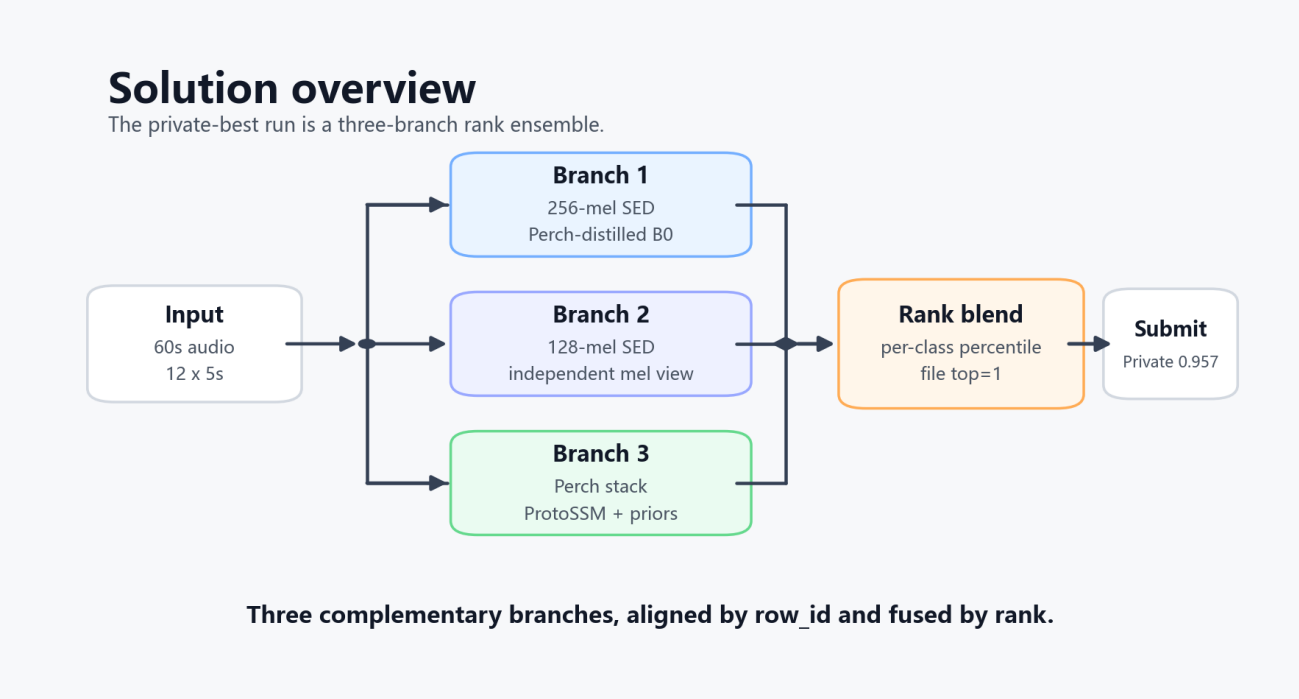
We would like to thank Kaggle and the BirdCLEF+2026 organizers for providing such an interesting competition, a well-run platform, and a dataset that made us listen more carefully to the natural world.

Submission	Description	Public LB	Private LB
`BirdCLEF+2026-infer - 957-5tta-model3`	3-model rank ensemble with TTA	0.954	0.957
`BirdCLEF+2026-infer - 4model-4321-192-923`	4-model rank ensemble	0.954	0.955

Overview

This solution was not a single magic trick. It was mostly a careful ensemble of several imperfect models. In BirdCLEF, different species, sites, time periods, and background textures favor different inductive biases, so we tried to combine complementary views instead of trusting one model too much.

The best private score came from the 3-model version. The 4-model version was still strong, but the additional NFNet branch did not improve private LB in this run.



Three complementary branches, aligned by row_id and fused by rank.

The private-best submission used three explicit branches: a 256-mel SED model, a 128-mel SED model, and the Perch/ProtoSSM stack. The 4-model experiment added an NFNet-style branch, but that was not the best private run.

Task Framing

Each soundscape is a 60 second recording. We split it into 12 windows of 5 seconds and predict 234 target classes for every window. All models use the `sample_submission.csv` order as the canonical class order.

There are two types of supervised data:

- `train_audio`: focal recordings with primary labels and secondary labels.

- `train_soundscapes_labels.csv`: labeled 5 second windows from soundscapes.

For validation, focal recordings are split with `StratifiedKFold` by primary label. Soundscape windows are split with `GroupKFold` by filename, so windows from the same 60 second file never leak across folds. We also track non-S22 macro AUC because S22 is noticeably noisy in the labeled soundscapes.

SED Models

The main CNN family is a sound event detection style model trained on 5 second audio chunks.

Audio is resampled to 32 kHz mono and converted to log-mel spectrograms. The core mel settings are:

Branch	Mel bins	Hop	Backbone
SED-1	256	512	EfficientNet-B0
SED-2	128	512	EfficientNetV2-S / EfficientNet-B0 style branch
SED-4	192	768	NFNet-L0

The SED head uses:

- a timm image backbone with one input channel,
- GeM pooling over the frequency axis,
- a 512-dimensional bottleneck,
- attention logits and class logits over the time axis,
- clip-level logits from attention-weighted frame logits.

During training, we supervise both the clip prediction and the strongest frame prediction:

```
loss = 0.5 * BCE(clip_logits, y) + 0.5 * BCE(max_frame_logits, y)
```

Some variants use focal BCE, label smoothing, class-frequency weighted sampling, background/no-call noise, SpecAugment, focal-focal MixUp, and focal-soundscape MixUp.

Perch Distillation

The 256-mel EfficientNet-B0 branch also uses Perch v2 as a teacher. A frozen Perch ONNX model produces a 1536-dimensional embedding for each 5 second chunk. That embedding is projected to 512 dimensions using a fixed random orthogonal projector, and the student backbone predicts it with an MSE distillation head.

This is useful because Perch carries broad bioacoustic knowledge that the small competition dataset cannot fully provide. The model still learns competition labels, but the representation is pulled toward a stronger acoustic embedding space.

Perch + ProtoSSM Branch

The third branch is intentionally very different from the mel CNNs.

For each 5 second window, Perch v2 produces:

- raw class scores,
- a 1536-dimensional embedding.

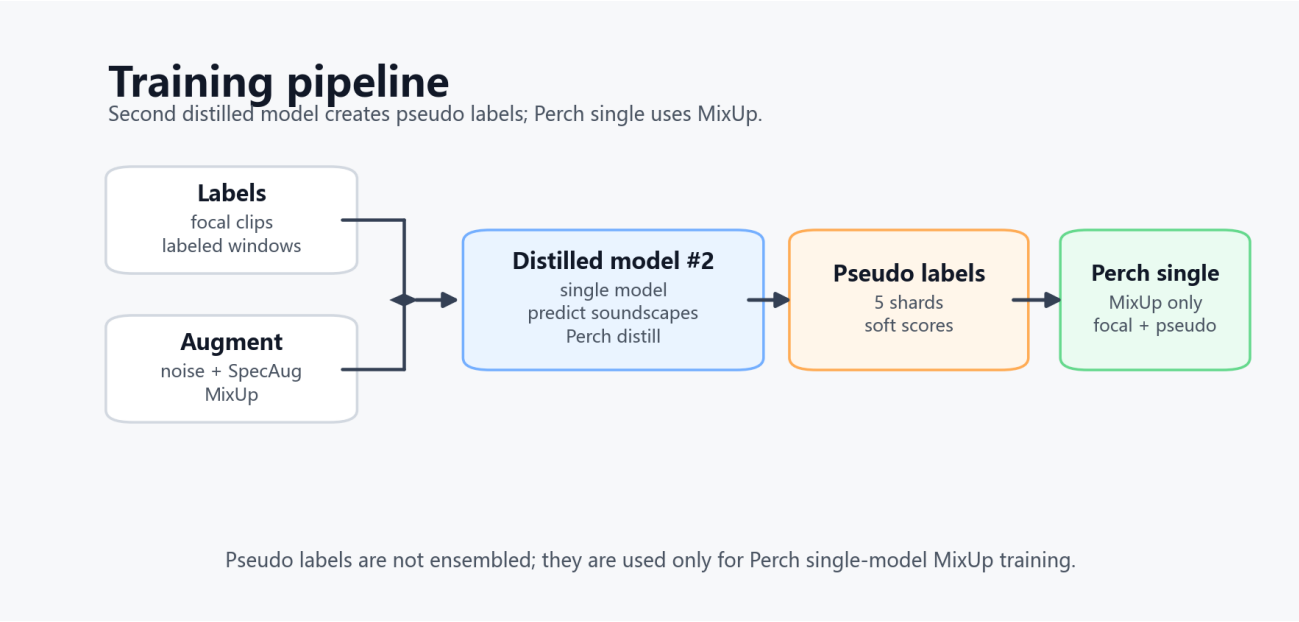
Then we add fold-safe metadata priors using site, hour, and site-hour tables. These priors looked useful for species with strong geography or time-of-day patterns. The prior fusion is taxonomy-aware, with different smoothing behavior for event-like classes and texture-like classes such as insects and amphibians.

The sequence model is ProtoSSM. It treats each 60 second file as a sequence of 12 windows and learns temporal corrections over Perch scores and embeddings. The main configuration is:

Component	Value
input embedding	1536
sequence length	12
model width	256
SSM layers	3
state dim	16
metadata	site + hour
extra heads	prototype head + family head

The branch also trains class-wise MLP probes on PCA-compressed Perch embeddings plus raw/prior/base scores. Finally, ProtoSSM and MLP/prior scores are blended, with a residual SSM correction available as a second pass.

Pseudo Labels



The pseudo-label part was narrower than the final ensemble:

1. Run the second distilled single model on unlabeled `train_soundscapes`.
2. Split the work into 5 shards to fit Kaggle runtime.
3. Merge the shards back into the canonical `row_id` and class order.
4. Use the resulting 5 second window scores as soft labels.

These pseudo labels were only used for the Perch single-model training run. The method itself stayed simple: pseudo MixUp mixes focal recordings with pseudo-labeled soundscape windows:

```
mixed_audio = 0.5 * focal_audio + 0.5 * pseudo_soundscape_audio
mixed_label = 0.5 * focal_label + 0.5 * pseudo_label
```

Before mixing, pseudo scores are sharpened with:

```
pseudo_label = score ** 1.6
```

The pseudo pool excludes validation files inside each fold, which keeps the loop cleaner. We did not use the full final ensemble to create these pseudo labels, and we did not use them as a general second-round signal for every branch.

Inference

Inference is designed to be reasonably CPU-friendly. The SED models are exported to OpenVINO XML and run in lots of 128 soundscape files. A shared audio cache and mel cache avoid repeated decoding and frontend work.

For each SED fold:

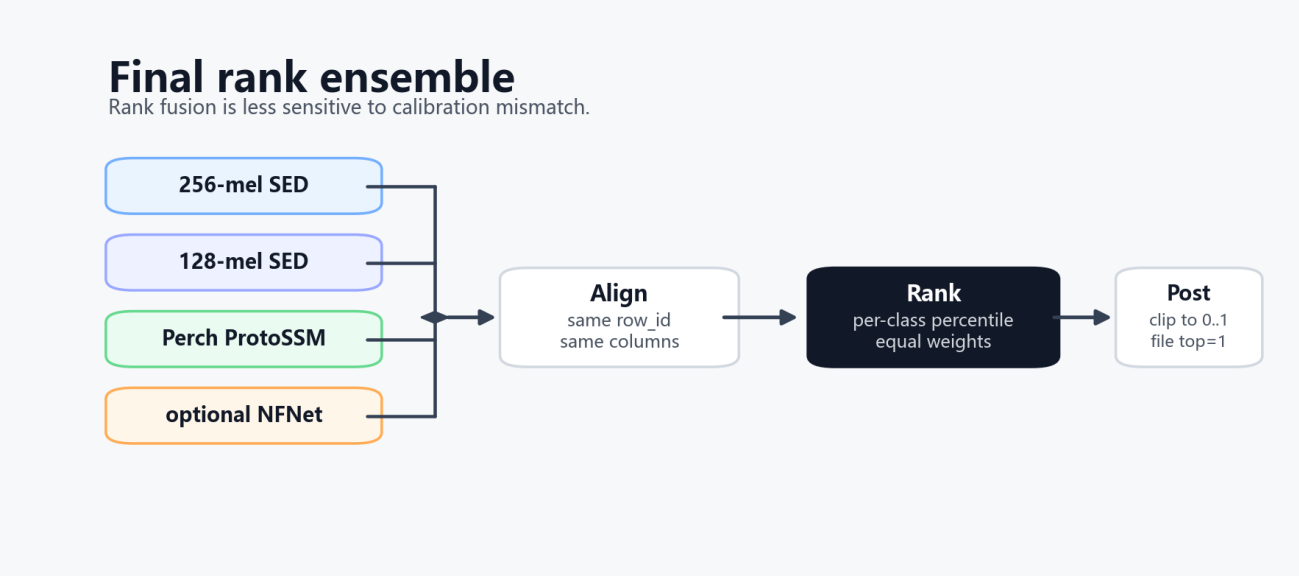
```
prediction = 0.5 * sigmoid(clip_logits) + 0.5 * sigmoid(max_frame_logits)
```

The fold predictions are averaged. Then we apply 3-point temporal smoothing:

Class group	Kernel
event-like classes	`[0.20, 0.60, 0.20]`
texture-like classes	`[0.35, 0.30, 0.35]`

The Perch/ProtoSSM branch keeps its own native post-processing: temperature scaling by taxon, file-level confidence scaling, rank-aware scaling, delta smoothing, and per-class threshold sharpening.

Rank Ensemble



The final blend is the part we trusted most.

Instead of averaging raw probabilities directly, we first align every model by `row_id`, then convert each model's prediction for each class into a global percentile rank:

```
ranked_score[class] = percentile_rank(model_score[class])
```

Then we average those ranks with equal weights.

This seemed to help because the branches have very different calibration:

- SED CNN probabilities are often sharp and local.
- Perch/ProtoSSM logits are more prior-aware and sequence-aware.
- The Perch single model trained with pseudo MixUp can change score scale.
- Different mel settings produce different confidence distributions.

Rank averaging preserves ordering while reducing calibration mismatch. After the rank blend, we apply one final file-level `top=1` postprocess: for each 60 second file and class, the strongest window scales the full file's 12 window predictions. This reinforces file-level presence while keeping window-level localization.

Ablation Study

We kept the ablation evidence modest and practical. Most rows below came from competition-side experiment logs rather than a single perfectly controlled offline protocol, so we treat them as directional engineering evidence.

For the V2S branch, the most useful changes were not one dramatic trick but a sequence of small decisions:

V2S experiment	LB	Note
initial V2S baseline	0.918	initial reference
clean-audio subset	0.918	private +0.001
moderate weighted sampling	0.924	WRS = 0.5; improved public LB
alternative weighted sampling	0.919	WRS = -1; private +0.004
file-level top-1 postprocess	0.926	strongest-window file scaling
full-audio training	0.929	broader audio usage
class-type temporal smoothing	0.933	event/texture-specific kernels
corrected V2S stack	0.942	baseline bug fixed; cumulative effect of the preceding changes

Class-type temporal smoothing uses different 3-point kernels for event-like and texture-like classes. A later removal check moved public and private in opposite directions: private improved by about `+0.002`, while public LB dropped by about `0.0005`.

For `corrected V2S stack`, we later found that the earlier baseline path had a bug. The large jump to `0.942` should therefore be interpreted as the cumulative result of the preceding changes after the baseline bug was corrected, not as a standalone weighting trick.

For the Perch/distillation side, pseudo labeling was more split-sensitive. It was not a clean public-LB gain, but it looked better on private in our logs:

Perch/distillation experiment	LB	Note
row-order alignment check	0.917 -> 0.914	alignment was a sensitive failure mode
512-dim Perch projection	0.917	private -0.004
Perch single without pseudo labels	0.926	stronger public LB
Perch single with pseudo-label MixUp	0.923	lower public, better private than no-pseudo in our log

What Worked

The main gains seemed to come from diversity, not from one huge model.

The ingredients that looked most useful were:

- multiple mel resolutions and backbones,
- clip plus frame-max supervision,
- Perch distillation for the 256-mel branch,
- Perch + ProtoSSM as a non-CNN sequence branch,
- site/hour priors,
- pseudo labels from the second distilled single model, used only for Perch single-model MixUp,
- global per-class rank fusion,
- final top-window file-level postprocessing.

The 4-model ensemble looked attractive, but the private LB favored the cleaner 3-model TTA ensemble. Our interpretation is that the extra branch added useful public-set diversity but also some private-set noise. Still, this is only a post-hoc read. Ensemble size is not the same thing as ensemble quality.

Inference Code

We plan to release the inference code here: open-source inference code.

Final Takeaway

To be honest, our result was largely built on last year's open-source code, along with the discussion posts and replies from many strong competitors. Those resources helped us a lot. Combined with the unavoidable role of luck, they were what allowed us to reach a gold medal.

Since we are still relatively new to this, we may not have presented every detail of our work. If you have any questions, please feel free to ask below.

The final score did not come from a single magic trick. It came from making a set of reasonable but not identical models disagree productively, then turning those disagreements into gains through rank fusion.

For our work this time, ensemble was basically our strongest weapon, and this time it was enough.

We would like to thank @tuckerarrants for the open-source code bc2026-distilled-sed, and @hideyukizushi for the open-source code bird26-reproduce-perch-protossm-resssm-inf-train.

We also want to thank @nikitababich for the BirdCLEF+ 2025 open-source solution 1st place solution: multi-iterative n, and @vladimirsydor and @vialactea for the BirdCLEF+ 2025 second-place writeup 2nd place journey down the rab.

Special thanks to @tuckerarrants for answering my questions in different comment sections. His replies helped me a great deal.